Last time: pointers.

Recall: pointers store memory addresses.

Mechanics / syntax

① Declare a pointer variable

```
int* p;
int *p;
int * p;
int*p;
```

all equivalent!

Note: can replace int with any other datatype

② get address of something, store in a pointer:

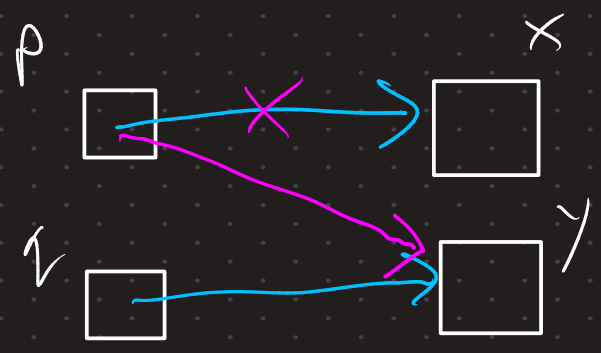
```
int x;
int* p = &x;   // & ≡ "address of" operator
```

③ Read/write the memory pointed to by a pointer:

```
cout << *p; // * = "dereference" operator.
*p = 77;
```

④ Assignment of pointers:

```
int x,y;
int *p, *q;
p = &x;
q = &y;
p = q;
```

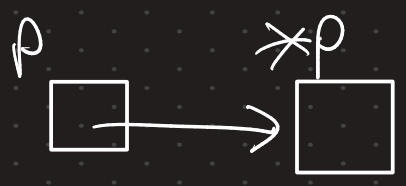


Dynamic memory

How to allocate (& deallocate) memory resources as your program is running.

Allocation:

```
int* p = new int;
```



"new" operator: ① allocates space
② tells you where to find it.

Deallocation

```
delete p;   // free memory pointed to by p.
```

"Memory leak": program which fails to deallocate memory it allocates.

Note: dynamic allocations will live in a different region of memory, known as the "heap" or "free store".

Pointer arithmetic

Say p is a pointer, i an integer.

Q: What is p+i?

A: another pointer, i "steps" away from p.

More precisely, p + (sizeof(*p) * i)

That is, each step is set to the size of the thing p points to.

Example: dynamic arrays:

```
int * p = new int[100];
```

<u>Fact:</u> $p[i] \equiv *(p+i)$

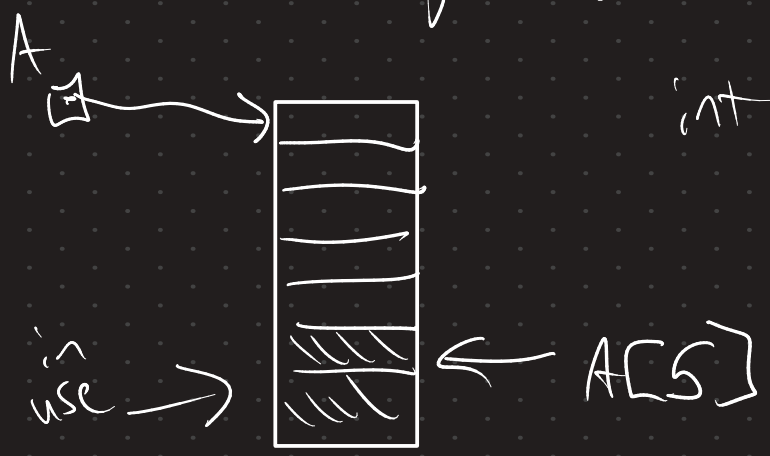
<u>Fun fact:</u> $p[i] \equiv *(p+i)$
$\equiv *(i+p)$
$\equiv i[p]$.

<u>Note:</u> to deallocate an array, add empty brackets:

```
delete[] p;
```

Exercise: resize a dynamically allocated array.

issue: space after last element might be in use!




```
int *A = new int[5];
```

Idea: have to move array to a larger home...